

Hong Kong Institute of Information Technology
Higher Diploma in Software Engineering (IT114105)
AY 2025/2026

ITP4510 Data Structures and Algorithms: Concepts & Implementation

ASSIGNMENT
Deadline: 12 Apr 2026 (Sun 23:59)

Hospital ER Triage Simulation

This is a simulation of a waiting line of patients at a hospital Emergency Room (ER). The purpose of the simulation is to provide a hospital manager with the means for predicting the effects of varying the number of doctors serving patients on the overall level of patient service.

Assume at least one doctor is ready for serving patients. The number of doctors can vary, and it stored in the variable **maxDoctors**. When a patient arrives at the ER and all doctors are busy treating other patients, the new arrival enters a single waiting line, which is a priority waiting queue called **waitLine**. The implementation of the waiting queue is using the **LinkedList** given in the laboratory exercise, and you must implement your own linked list (do not use Java API data structure classes). For simplicity, we assume that no more than one patient can arrive at the ER during the same minute (that means in each minute only have ONE patient come or none).

Before initiating the simulation, it needs to input the duration of the simulation in minutes and specify the number of doctors to be on duty. Once the simulation commences, for every minute, you will be prompted to enter the triage level for any arriving patient (assuming a maximum of one patient per minute). In case no patient arrives within a minute, then input a value of 0. If a patient arrives (triage level is 1, 2 or 3), then you will be prompted to enter the patient name and the treatment time (in minutes).

The waiting queue must follow strict triage priority and FIFO order:

- Level 1 (Critical) is inserted before all Level 2 and Level 3 patients.
- Level 2 (Urgent) is inserted after all Level 1 and existing Level 2 patients, but before Level 3 patients.
- Level 3 (Normal) is inserted at the tail.

The program should provide the statistical data after simulation completed, for example:

- How many minutes of the simulation?
- How many doctors exist in the simulation?
- The total number of the patients has served.
- How many patients wait in the queue in average? (Average Waiting Line Length)
- Maximum queue length during the simulation.

Sample of simulation

Here is a sample of one simulation. User inputs are in bold and underlined.

```
----- SETUP SIMULATION ENVIRONMENT-----
Input simulation length (min): 6
Input number of doctors: 2

----- START SIMULATION -----

At the beginning of iteration 1 ...
Input triage level for a new patient [1-Critical,2-Urgent,3-Normal,0-None]: 3
Input patient name: Alice
Input treatment time (min): 5
After 1 minute ##
    Doctor_1[ 6 ]    Doctor_2[ 0 ]    Waiting Queue: [ ]

At the beginning of iteration 2 ...
Input triage level for a new patient [1-Critical,2-Urgent,3-Normal,0-None]: 3
Input patient name: Bob
Input treatment time (min): 5
After 2 minute ##
    Doctor_1[ 6 ]    Doctor_2[ 7 ]    Waiting Queue: [ ]

At the beginning of iteration 3 ...
Input triage level for a new patient [1-Critical,2-Urgent,3-Normal,0-None]: 3
Input patient name: Charlie
Input treatment time (min): 3
After 3 minute ##
    Doctor_1[ 6 ]    Doctor_2[ 7 ]    Waiting Queue: [ (Charlie,L3,3) ]

At the beginning of iteration 4 ...
Input triage level for a new patient [1-Critical,2-Urgent,3-Normal,0-None]: 1
Input patient name: David
Input treatment time (min): 4
After 4 minute ##
    Doctor_1[ 6 ]    Doctor_2[ 7 ]    Waiting Queue: [ (David,L1,4)
(Charlie,L3,3) ]

At the beginning of iteration 5 ...
Input triage level for a new patient [1-Critical,2-Urgent,3-Normal,0-None]: 2
Input patient name: Eve
Input treatment time (min): 3
After 5 minute ##
    Doctor_1[ 6 ]    Doctor_2[ 7 ]    Waiting Queue: [ (David,L1,4) (Eve,L2,3)
(Charlie,L3,3) ]

At the beginning of iteration 6 ...
Input triage level for a new patient [1-Critical,2-Urgent,3-Normal,0-None]: 0
After 6 minute ##
    Doctor_1[ 10 ]    Doctor_2[ 7 ]    Waiting Queue: [ (Eve,L2,3)
(Charlie,L3,3) ]

----- END OF SIMULATION -----
Total minute simulated: 6 minutes
Number of Doctors: 2
Number of patient served: 3 patients
Average Waiting Line Length: 1.3333334 patients
Maximum Waiting Line Length: 3 patients
-----
```

Program Logic

(One iteration/cycle represents one minute)

```
----- SETUP SIMULATION ENVIRONMENT-----
Input simulation length (min): 6
Input number of doctors: 2
```

Iteration	Event Happened
#1	At the beginning of iteration 1 ... Input triage level: 3 Input name: Alice , treatment time: 5 → A new patient arrived (Lvl 3). → Doctor_1 is available, patient goes to Doctor_1. → Leave time = 1 + 5 = 6
	After 1 minute ## Doctor_1[6] Doctor_2[0] Waiting Queue: []
#2	At the beginning of iteration 2 ... Input triage level: 3 Input name: Bob , treatment time: 5 → A new patient arrived (Lvl 3). → Doctor_2 is available, patient goes to Doctor_2. → Leave time = 2 + 5 = 7
	After 2 minute ## Doctor_1[6] Doctor_2[7] Waiting Queue: []
#3	At the beginning of iteration 3 ... Input triage level: 3 Input name: Charlie , treatment time: 3 → Both doctors are busy, patient enters waiting queue (Lvl 3 goes to tail).
	After 3 minute ## Doctor_1[6] Doctor_2[7] Waiting Queue: [(Charlie,L3,3)]
#4	At the beginning of iteration 4 ... Input triage level: 1 Input name: David , treatment time: 4 → Both doctors are busy, patient enters waiting queue. → Lvl 1 has highest priority, inserted before Lvl 3.
	After 4 minute ## Doctor_1[6] Doctor_2[7] Waiting Queue: [(David,L1,4) (Charlie,L3,3)]

#5	At the beginning of iteration 5 ... Input triage level: 2 Input name: Eve , treatment time: 3 → Both doctors are busy, patient enters waiting queue. → Lvl 2 inserted after all Lvl 1 and existing Lvl 2, before Lvl 3.
	After 5 minute ## Doctor_1[6] Doctor_2[7] Waiting Queue: [(David,L1,4) (Eve,L2,3) (Charlie,L3,3)]
#6	At the beginning of iteration 6 ... Input triage level: 0 → value of 0 that means no patient came. → Patient in Doctor_1 has left at this minute (leaveTime=6), Doctor_1 becomes IDLE. → Doctor_1 takes the first patient in queue: David (Lvl 1). → Leave time = 6 + 4 = 10
	After 6 minute ## Doctor_1[10] Doctor_2[7] Waiting Queue: [(Eve,L2,3) (Charlie,L3,3)]

Final result

Total minute simulated: 6 minutes
 Number of Doctors: 2
 Number of patient served: 3 patients

Provided Files (Given Files)

The following Java files will be provided to students: LinkedList.java and ListQueue.java.
 Students are required to use the given linked list / queue implementation in this assignment (i.e., the waiting line waitLine must be implemented based on the given LinkedList/ListQueue).

- You are **NOT ALLOWED** to use data structure classes provided by Java API (e.g. Java API LinkedList / Queue).
- You may create **new classes** in your solution (e.g. to support triage priority insertion) as needed, but the waiting line must be based on the given files.

Task Specification

1. Correct use of Priority Queue using linked list (priority insertion + FIFO within same triage).
2. Correct use of Queue class using the given files (LinkedList / ListQueue).
3. Implementation of the program using Java. **Submit listings of all programs.**
4. Exceptional/abnormal cases. You should create your own Exception class. Submit a brief description of all such cases (e.g. **invalid input**) handled by the program.
5. Program structure and in-program comments.
6. Evidence of testing. Test the program and submit the logged **listing of run samples / screen dumps.**
7. The startup JAVA that contains the main method MUST be called as **ERSimulation.java**

Mark Allocation

0. Compilation -- success compilation and execute the program	10%
1. Design -- correct using Queue	10%
2. Implementation -- input parameter -- program simulation -- result print out	40%
3. Statistical data and extra functions provided and	10%
4. Error handling -- create own Exception class -- handle exception	10%
5. Report -- executable results (screen dumps)	10%
6. Coding standard -- proper indentation -- proper naming -- consistency coding style -- appropriate comments	10%
Total	100%

**** This assignment is counted for your EA. The weighting of this assignment is 40% of EA.**

Instructions to Students

This assignment is an individual assignment. Each student has to submit his/her own work. Plagiarism will be treated seriously. All assignments that have been found involved wholly or partly in plagiarism (no matter these assignments are from the original authors or from the plagiarists) will score ZERO marks. Further, disciplinary action will be followed.

Adequate in-program comments should be placed as appropriate. All user-defined names should be descriptive as much as possible. Marks are given based on correctness, programming quality, and style.

You are required to hand in:

- All classes (programs) written and well commented by you for this assignment.
- The evidence of testing - You should submit at least 4 dump screens (similar to the sample output given above) to show the run results with your own input files (that means you should design your own test cases).

All work is to be submitted to Moodle on/before **12th April 2026 11:59 pm**. Late submission without valid reasons will be given ZERO mark.